AIMERZ.ai
Aim.Act.Achieve

NOTES

Working with Modules & Python Package Management

1. Working with Modules and Python Package Management

Definition: Module and Package

- A **module** in Python is a single .py file containing code such as functions, classes, or variables that can be reused in other programs.
- A **package** is a directory that contains multiple related modules and an __init__.py file. This file indicates to Python that the directory should be treated as a package.

Importance of Packages for Large Projects

Packages provide structure to large projects by grouping related modules under one directory, making it easier to manage, debug, and collaborate on code. Packages enable modularity, where:

- **Code reuse** is encouraged as modules can be imported and reused.
- **Easier debugging** allows developers to isolate issues in specific modules.
- **Collaboration** becomes smoother in team environments, as different developers can focus on different modules.

Why Packages Help Organize Large Projects

Packages provide a clear hierarchy and organization to your project. Imagine a large-scale application like a web framework or data analysis tool that requires multiple functionalities—packages help break these down into smaller, manageable modules. This modularity:

- Promotes **code reuse**,
- Simplifies **debugging** by isolating issues to specific modules,
- Enables better **collaboration** in larger teams, as different developers can work on different parts of the project.

2. Modular Programming

Definition: Modular Programming

- Modular programming is a technique where code is divided into separate, independent modules.
- Each module is responsible for a specific part of the functionality, which allows for more organized, reusable, and maintainable code.

Structuring Large Programs with Modules

A module can be created by saving Python code to a .py file and using the import statement to use its functionality in other scripts.

A. Creating a Simple Module

- Definition of a Module
  - A **module** is simply a file containing Python code (with a .py extension).
  - Modules can store functions, classes, and variables, and they help you organize code into reusable pieces.

Steps to Create and Use a Module

a. Create a New Python File

- Make a file named add.py.
- Inside add.py, write a function that performs a simple task, like adding two numbers.

File :add.py

```
# add.py
def add(a, b):
    return a + b
```

b. Import and Use the Module in Another Script

- Create a new script file, main.py.
- Use the import statement to bring in the add module and access its add function.

```
#main.py
import add

result = add.add(3, 4)
print(result)  # Output: 7
```

Explanation:

- import add imports the add.py file.
- add.add(3, 4) calls the add function inside the add module.

- Running main.py will output 7, demonstrating that the module is successfully used.

B. Organizing Modules into a Package

Definition of a Package

- A **package** is a collection of related modules organized in a directory with an `__init__.py` file.
- The `__init__.py` file turns the directory into a Python package, allowing you to import modules from that directory.

Steps to Create a Package

Suppose you want to create a `math_operations` package with different mathematical operations (like addition and subtraction) as modules.

1. **Set Up the Directory Structure**
   - Create a folder named `math_operations`.
   - Inside `math_operations`, create an empty `__init__.py` file. This tells Python that `math_operations` is a package.

     Directory Structure:

     > math_operations/
     >
     > __init__.py
     >
     > add.py
     >
     > subtract.py

2. **Create Modules in the Package**
   - Inside math_operations, create modules like add.py and subtract.py.

File: add.py

```
def add(a, b):
    return a + b
```

File: subtract.py

```
def subtract(a, b):
    return a - b
```

3. **Importing and Using the Package in Another Script**
    - Now, create a script file `main.py` outside of the `math_operations` package.
    - Import functions from the `math_operations` package using the syntax `from package.module import function`.

File: `main.py`

```
from math_operations.add import add
from math_operations.subtract import subtract

print(add(5, 3))       # Output: 8
print(subtract(10, 4))  # Output: 6
```

Explanation:

- `from math_operations.add import add` imports the `add` function from `add.py` inside the `math_operations` package.
- Similarly, `from math_operations.subtract import subtract` imports the `subtract` function.

Running `main.py` will display:

```
8
6
```

C. Understanding `__init__.py` in a Package

Purpose of `__init__.py`

The `__init__.py` file:

- Allows the directory to be treated as a package.
- Can specify which modules or functions are available for import when the package is imported.

Customizing `__init__.py`

To make certain functions directly accessible when you import the package, you can define them in `__init__.py`.

1.Modify `__init__.py`

Update `__init__.py` to import the main functions you want to be accessible from the package level.

File: `__init__.py`

```
from .add import add
from .subtract import subtract
```

2. Using the Package with `__init__.py` Customization

Now you can directly import `add` and `subtract` from `math_operations`.

File: `main.py`

```
from math_operations import add, subtract

print(add(5, 3))       # Output: 8
print(subtract(10, 4))  # Output: 6
```

Explanation:

By defining imports in `__init__.py`, the functions `add` and `subtract` are accessible directly from the package, making imports simpler and more intuitive.

3. Using External Libraries

Definition: External Libraries

External libraries are pre-packaged modules and packages created by the Python community and available on the Python Package Index (PyPI). They provide pre-built solutions for a variety of tasks, such as data analysis or web development.

Installing Third-Party Libraries with `pip`

You can use `pip` to install libraries from PyPI directly into your environment.

Example: Installing Libraries

Install a commonly used library, `requests`, which makes handling HTTP requests easy:

```
pip install requests
```

Using Installed Libraries

Once a library is installed, you can import and use it in your Python code.

Example: Using the `requests` Library

```
import requests

response = requests.get('https://jsonplaceholder.typicode.com/todos')
todos = response.json()

for todo in todos[:5]:  # Display first 5 tasks
    print(f"{todo['id']}: {todo['title']}")
```

Troubleshooting Installation Issues

If you encounter issues with package installations, here are some steps to resolve them:

1. **Ensure `pip` is up-to-date:**

   Run the following command to upgrade `pip`:

   ```bash pip install --upgrade pip

## 4. Virtual Environments

### Definition: Virtual Environment

A **virtual environment** is an isolated Python environment within a directory. It includes its own Python interpreter and package directories, allowing projects to manage dependencies independently.

### Why Virtual Environments Are Essential

Virtual environments prevent dependency conflicts by isolating a project's packages. They allow multiple projects to coexist on the same system without interfering with each other's dependencies.

---

### Example: Creating and Using a Virtual Environment

1. **Create a virtual environment:**

```
python -m venv myenv
```
```

Create a virtual environment:

```
python -m venv myenv
```

2. **Activate** the virtual environment:
    - **Windows**

      myenv\Scripts\activate
    - **macOS/Linux**

      myenv\Scripts\activate

3. **Install packages in the virtual environment (e.g., `requests`):**

```
pip install requests
```

4. **Install packages in the virtual environment (e.g., `requests`):**

```
deactivate
```

Example Usage: Managing Dependencies with `requirements.txt`

1. **Generate a `requirements.txt` file** to list all project dependencies:

```
pip freeze > requirements.txt
```

5. Best Practices in Imports

Organizing Imports

To maintain readability and organization, follow these guidelines:

1. **Standard library imports** (e.g., `import os`).
2. **Third-party library imports** (e.g., `import requests`).
3. **Local module imports** (e.g., `from mymodule import add`).

This order ensures clarity and allows developers to quickly distinguish between built-in, external, and project-specific imports.

Avoiding Circular Imports

Circular imports occur when two or more modules import each other, causing a deadlock. To avoid circular imports:

- Refactor code so that no module directly depends on another.
- Move shared functionality to a common module if possible.

Example: Resolving Circular Imports

If `moduleA.py` imports `moduleB.py`, and `moduleB.py` imports `moduleA.py`, refactor the code so that both modules depend on a third module, `moduleC.py`, which contains the shared functionality.

Using `__init__.py`

The `__init__.py` file makes a directory importable as a package. It can also be used to initialize or expose specific classes and functions from within the package, making them accessible directly from the package import.

Example: Setting Up `__init__.py`

File: `math_operations/__init__.py`

```
from .add import add
from .subtract import subtract
```

Now you can import directly from `math_operations`:

```
from math_operations import add, subtract

print(add(5, 3))       # Output: 8
print(subtract(10, 4))  # Output: 6
```

6. Creating and Distributing Packages

Definition: Package Distribution

Package distribution allows developers to share their code on platforms like PyPI. This requires a `setup.py` file that describes the package and its dependencies.

Basics of Packaging with `setuptools`

`setuptools` is a library used to create and distribute packages in Python. It helps with packaging code, managing dependencies, and creating the necessary files to distribute your code.

Example: Creating `setup.py`

Define package metadata and dependencies in `setup.py`:

File: `setup.py`

```
from setuptools import setup, find_packages

setup(
    name='math_operations',               # Package name
    version='0.1',                        # Initial package version
    description='A simple math operations package',  # Short description
    author='Your Name',                   # Your name
    author_email='your.email@example.com', # Your email
    packages=find_packages(),             # Automatically find all packages
    install_requires=[                    # List of dependencies
        'requests',                       # Example external dependency
    ],
)
```

Example: Directory Structure for Distribution

Suppose you have a package called `mypackage` with a module, `fetch_user.py`.

Directory Structure:

> mypackage/
>
> `__init__.py`
>
> fetch_user.py
>
> setup.py

File: `fetch_user.py`

```
import requests

def get_user(id):
    response = requests.get(f'https://jsonplaceholder.typicode.com/users/{id}')
    return response.json()
```

After uploading, others can install your package via `pip`:

```
pip install mypackage
```

THANK YOU

AIMERZ.ai
Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan